PSC-OS

PSC-ONE Operating System

実装仕様書

Rev.0.9 — Preliminary　　2026 年 10 月 10 日

QPSC-Design　　ソース・技術資料・画像の統合版



図 1: PSC-ONE ボードの外観。既存のボード写真を収録。

概要

PSC-OS は、PSC-ONE の RISC-V CPU、Sv32 方式のアドレス変換、SDRAM、周辺回路および SynapEngine を利用・検証するための独自 OS である。M-mode の BootROM ／ MBIOS、S-mode カーネル、U-mode シェルを組み合わせ、UART から機器操作とアプリケーション実行を行う。

full shell には FAT32、JPEG デコーダ、MicroPython、INT8 fully connected 推論環境が組み込まれる。外部 ELF32 を SD カードから読み込み、終了または処理可能なユーザー例外の後にシェルへ戻る構成を備える。

対象	本書の基準
主対象	PSC 構成・CPU v1・通常の full shell
ソフトウェア ABI	RV32、ILP32。既定ビルドは `rv32im_zicsr_zifencei`
ユーザー環境	1 MiB の仮想領域、UART シェル、単一 foreground ELF
起動方式	direct boot、SD FAT32 boot、別構成として raw-sector ／ SBI
位置付け	教育・研究、CPU ／ OS ／アクセラレータ開発、組込み機能評価

本書は 2026 年 10 月 10 日にローカルの `PSC-ONE` を調査した実装仕様である。収録画像と過去の試験報告を、新たな実機動作確認や性能保証として扱わない。

根拠：[1] OS 全体資料、[2] OS Makefile、[18] 画像・ライセンス。

PRELIMINARY

目次

1 ソフトウェア構成と実行層 **3**

2 起動シーケンスと配布イメージ **4**

3 メモリ配置と容量管理 **5**

4 Sv32 変換と保護機能の範囲 **6**

5 タスク・例外・タイマー割込み **7**

6 システムコール ABI と基本サービス **8**

7 デバイス系システムコール **9**

8 UART シェルと操作体系 **10**

9 SD カードと FAT32 ファイルシステム **11**

10 外部 ELF32 アプリケーション **12**

11 周辺デバイスと時間・音声サービス **13**

12 DMA 転送と SynapEngine API **14**

13 JPEG デコードと LCD 表示 **15**

14 TFLite INT8 推論ランタイム **16**

15 推論 backend 選択と性能資料の解釈 **17**

16 MicroPython 組込み環境 **18**

17 ビルド構成と検証手順 **19**

18 実装範囲・参照資料・改訂履歴 **20**

本書の読み方

アドレスは特記がない限り 16 進数、KiB ／ MiB は 2 進単位とする。PA は物理アドレス、VA は仮想アドレス、PTE はページテーブルエントリを表す。コマンド例の作業位置は各節に示す。

コード、公開ヘッダ、リンカスクリプトを現行仕様の根拠とし、Markdown は構成・背景・試験条件の補足に用いた。資料間の相違は現行コードを優先する。特に TFLite の初期 Phase 資料、旧 LCD 設定、過去のメモリ使用量は、そのまま現行仕様へ転記していない。

本文の [1]〜[18] は最終ページの参照資料に対応する。詳細な参照パスと SHA-256 は同梱の `PSC_OS_Specs.sources.txt` に記録する。画像は `figures/` に複製し、文書のビルド時に元リポジトリへの絶対パスを必要としない。

PRELIMINARY

1 ソフトウェア構成と実行層

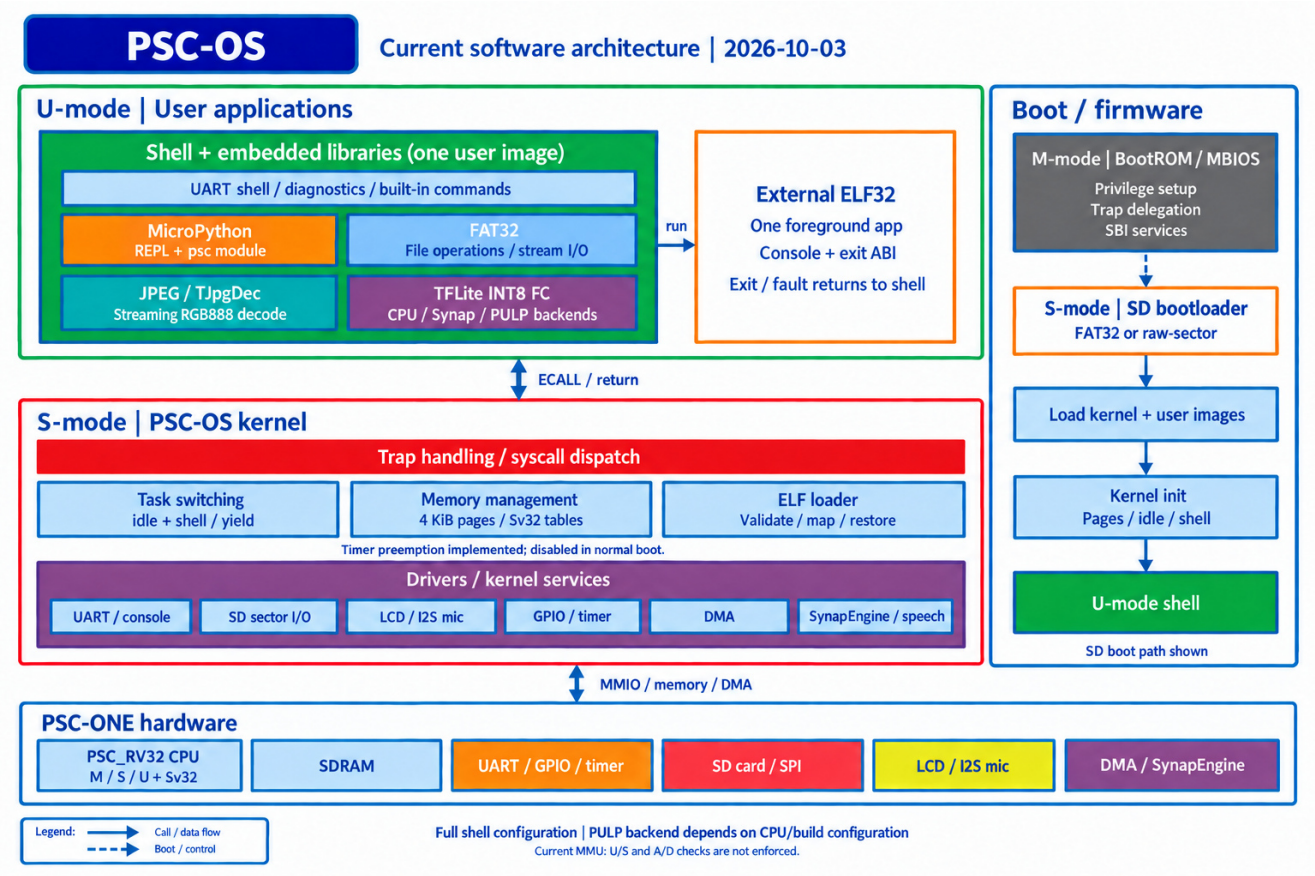


図 2: full shell のソフトウェア構成図（原図の日付：2026-10-03）。

実行層	主な実装	責務
M-mode	BootROM / MBIOS	特権初期化、例外委譲、S-mode への移行、Machine timer trap 設定。
S-mode	SD bootloader	SD からカーネルとユーザーイメージをロード。
S-mode	kernel / drivers	ページ確保、タスク context、trap、syscall、デバイス操作、ELF 配置。
U-mode	shell / libraries	コマンド解析、FAT32、JPEG、MicroPython、TFLite。デバイスアクセスは ECALL 経由。
U-mode	外部 ELF	専用の一時アドレス空間で同期実行。公開 ABI はコンソールと終了に限定。

コンポーネントの境界

FAT32 や TFLite はカーネルの常駐サービスではなく、シェルのユーザーイメージにリンクされる。したがって、外部 ELF へ同じ機能が自動的に公開されるわけではない。MicroPython もシェルと同じイメージ内で動作し、TFLite の resident model を共有する。

ソースは `software/os/src/` の `boot/`、`kernel/`、`shell/`、`drivers/`、`fs/`、`api/`、`lib/` に分かれる。OS Makefile はそれぞれの実行層に必要なソースを選択してリンクする。

根拠：[1] `docs/psc_os.md`、[2] Makefile、[18] `docs/images/PSC_OS.jpg`。

PRELIM

2 起動シーケンスと配布イメージ

FAT32 boot の処理順序

1. BootROM は M-mode でスタックと `mtvec` を初期化し、`satp=0` でアドレス変換を無効にする。
2. `medeleg=0xb1ff` で同期例外と U-mode ECALL を委譲する。S-mode ECALL は MBIOS へ残す。
3. `USE_BOOTLOADER` 構成では `mepc=0x00100000`、direct 構成では `0x00200000` を設定し、`mret` で S-mode へ移る。
4. FAT32 bootloader が SD を初期化し、ルートの `KERNEL.MEM` と `USER.MEM` を読み込む。ロード先はそれぞれ `0x00200000`、`0x00400000`。
5. カーネルが BSS、trap 入口、idle ／ shell、ページテーブルを準備する。シェルを実行用ページにコピーし、コードをキャッシュへ反映する。
6. `yield()` で shell へ切り替え、`sret` で U-mode へ進む。UART の `PSC_OS>` を起動完了の目安とする。

成果物	用途と取扱い
`bootrom.mem`	M-mode 初期化用。FPGA ／シミュレータ側の ROM 構成と整合させる。
`bootloader_fat32.mem`	FAT32 SD 起動用のローダイメージ。
`kernel.mem` / `user.mem`	同じビルドの 2 ファイルを SD ルートへ大文字名で配置。
`*.ELF`	起動後の `run` で読み込むアプリケーション。OS 起動用 MEM とは別形式。

FAT32 boot の MEM はテキスト形式である。`.bin` を単に改名して使用しない。raw-sector 版の IMG 配置とも互換ではない。BootROM・bootloader・kernel・user を同一構成へそろえる。



図 3: 既存資料にある LCD 起動画面。表示内容は撮影時の構成に依存する。

現行 `kernel.c` は `NPU_TESTMODE=1`、`PSC_LOGOMODE=0` である。写真にあるロゴが通常カーネルで必ず表示される、という仕様ではない。SD 読出しには CRC 照合と最大 3 回の読出し試行があり、ロード失敗時は失敗箇所を UART へ出力する。

根拠：[3] `src/boot/psc_mbios.S`、`bootloader_fat32.c`、`boot_fat32.c`、[4] `kernel.c`。

3 メモリ配置と容量管理

既定 PSC 構成のアドレス

アドレス／範囲	種別	内容
0x00000000	起動側領域	BootROM 配置の基点。リンカの BRAM 領域定義は 64 KiB。
0x00020000	NPU 用 PA	SynapEngine データ領域。
0x00030000	NPU 用 PA	結果領域。最大 64×64 の INT32 結果で 16 KiB。
0x00100000	PA	SD bootloader の実行基点。
0x00200000	PA ／恒等 VA	カーネルのリンク・ロード基点。
__free_ram から	PA	4 KiB 境界のページ確保領域。上限は後述の式。
0x00400000	ロード元 PA	外部ロードするシェルイメージの配置先。
0x00400000〜0x004fffff	U-mode VA	既定ユーザーウィンドウ。実行時 PA はカーネルが別途確保。
0x004e0000〜0x004fffff	shell VA	通常シェル用に別途マップする 128 KiB スタック。
0x10000000〜0x1000ffff	MMIO	カーネルが R/W で恒等マップする周辺領域。

物理ロード先と実行 VA の関係

シェルのロード元 PA とユーザー VA の基点は同じ数値であるが、実行時のページは alloc_pages() で確保する。ユーザー VA をそのまま DMA や NPU へ渡す設計ではない。カーネル領域は __kernel_base から __free_ram_end まで恒等マップする。

kernel_fpga.ld はカーネル本体と BSS の後に 128 KiB の起動用スタックを予約する。ページプールは次式で制限し、シェルのコピー元を上書きしない。

$$P_{end} = \min(P_{start} + 1\,\text{MiB},\ \text{shell ロード元 PA})$$

さらにリンク時 ASSERT で「4 KiB 単位へ切り上げたシェルコピー＋ 128 KiB スタック＋ページテーブル予算 32 KiB」がプール内に収まることを確認する。プールの開始位置や残容量は、kernel.map と shell.map でビルドごとに確認する。

確保方式とスタックの区別

alloc_pages(n) は先頭から前進する単純なアロケータで、確保ページをゼロ初期化する。上限超過は PANIC となる。一般的なページ解放、swap、demand paging はない。プロセスごとの trap 用 8 KiB、Machine IRQ 用 4 KiB、外部 ELF 用 16 KiB は、通常シェルの 128 KiB スタックとは別である。

user.ld には BSS 内にも guard ／ stack 予約があるが、create_process() はウィンドウ末尾へ別の実行スタックをマップする。リンカの guard という名称だけから未マップ保護を保証しない。外部 ELF の未マップ guard は第 10 節の配置による。

根拠：[4] kernel 関連リンカ、user_memory.h、kernel_process.c、[5] src/shell/user.ld。

PRELIMINARY

4 Sv32 変換と保護機能の範囲

ページテーブル

ユーザー領域は 4 KiB 単位でマップする。1 ページのテーブルは 32-bit PTE を 1,024 個格納し、VA を VPN[1]、VPN[0]、offset に分けて 2 段で参照する。

$$\underbrace{VA[31:22]}_{VPN[1]:10bit} \quad \underbrace{VA[21:12]}_{VPN[0]:10bit} \quad \underbrace{VA[11:0]}_{offset:12bit}$$

項目	設定／判定	実装上の意味
SATP.MODE	bit 31	0 は Bare、1 は Sv32 変換経路。M-mode は変換をバイパス。
SATP.PPN	bits 21:0	ルートテーブル PA のページ番号。
PTE.V	bit 0	0 は不正エントリ。
PTE.R/W/X	bits 1/2/3	LOAD ／ STORE ／命令 fetch の許可を確認。W=1 かつ R=0 は不正。
PTE.U	bit 4	OS は user ／ kernel で設定を区別するが、現行 MMU は U/S 検査を省略。
PTE.A/D	bits 6/7	現行 MMU は検査・自動更新を省略。
上位 leaf	4 MiB ページ	PPN[0]=0 と権限を確認。OS の通常ユーザー配置は 4 KiB。

アドレス空間切替

`yield()` は次プロセスの root PPN へ `satp` を切り替え、その前後に `sfence.vma` を実行する。現行 MMU の PTE キャッシュは ASID 別の管理を行わないため、ASID を独立した隔離・キャッシュ識別機構として用いることはできない。

命令 fetch にも VA から PA への変換が必要である。シェルや ELF のコードを別 PA へコピーした後、`sync_user_code()` が D-cache 書戻し、直列ロードによる完了待ち、`fence.i` を行う。単にページテーブルを書き換えるだけでは、古い命令や未反映のコードを実行する可能性がある。

ソフトウェアによるバッファ検査

`user_accessible()` はアドレス範囲、加算のあふれを避けたサイズ判定、各 4 KiB ページの V/U と必要な R/W を調べる。ELF 入力、NPU の入出力、LCD 矩形転送で使用する。一方、既存の dump や一部 SD サービスは同じ検査を備えないため、全 syscall が同一水準で検査されるとは記述しない。

現行 v1 MMU は R/W/X を検査するが、U/S および A/D は検査しない。U=0 のカーネル／ MMIO マッピングを作ることだけでは、信頼できない U-mode コードを隔離できない。ELF の形式検査・syscall 制限もこのハードウェア制約を補完するものではない。

適用範囲

MMU 内部には 34-bit アドレスがあるが、外部 PA ／ PTE 読出しアドレスは下位 32 bit を使う。4 GiB 以上の物理領域、完全な Sv32 特権保護、一般的な仮想記憶 OS の全機能を保証する仕様ではない。legacy ／ v2 の利用時は対応 RTL と回帰結果を個別に確認する。

根拠：[6] `docs/cpu_mmu.md`、v1 `MMU.sv`、[4] `kernel.c`、[7] `kernel_syscall.c`。

PRELIMINARY

5 タスク・例外・タイマー割込み

プロセス構造と通常起動

`structprocess` は PID、状態、保存 SP、Machine context、ページテーブル、8 KiB のカーネルスタックを保持する。スロット上限は `PROCS_MAX=8`、状態値は UNUSED=0、RUNNABLE=1、EXITED=2 である。これは内部管理上の上限であり、通常起動で 8 個のユーザーアプリケーションを同時に起動するインターフェースがあることを意味しない。

通常の `kernel_main()` は idle と shell を生成し、`yield()` で実行を移す。`MULTITASK_MODE=0` のため、追加 kernel task の生成と `preemption_start()` は通常経路で無効である。協調切替は実行可能なタスクを走査して選び、SATP、`sscratch`、callee-saved レジスタと戻り先を切り替える。

U-mode trap の処理

1. ユーザー SP を保存し、プロセス固有のカーネルスタックへ切り替える。未マップのユーザー SP を trap 保存先として使用しない。
2. 整数レジスタと SEPC、SSTATUS、SCAUSE、STVAL を trap frame へ保存する。
3. U-mode ECALL（cause=8）は `handle_syscall()` へ配送する。復帰 PC を次命令へ進め、サービスによって戻りレジスタを更新する。
4. 外部 ELF の処理可能な fault は cause ／ PC ／ address を出力し、保存済みシェル context へ戻す。その他の異常は既存の trap ／ PANIC 処理に従う。
5. trap frame から復元して `sret` する。ELF 起動時は SEPC・SP・SSTATUS も新しい context へ置き換える。

プリエンプション経路

Machine timer IRQ は `mcause=0x80000007` で認識する。`preemption_start()` は MBIOS extension 3 を使って Machine trap 入口と専用スタックを登録し、1 ms の scheduler tick を開始する。`timer_start_scheduler_tick()` の reload は 999 である。

`machine_context` は整数レジスタに加えて MEPC、MSTATUS、MCAUSE、SSCRATCH、Supervisor trap CSR を保存する。S-mode 処理やスタック切替の途中で割込みを受けても、その context へ戻れる構成である。プリエンプション有効時の `yield()` は extension 6 の ECALL 経路を使う。

区分	意味
通常起動	idle ／ shell、明示的 `yield()`、追加タスクの周期切替は無効。
専用マルチタスク試験	Machine IRQ、レジスタ保存、複数タスクの再開を確認する構成。
時間計測	同じ timer 資源を使用するため、使用中なら計測 API は利用不能を返す。
提供しない機能	POSIX process、fork、一般的 execve、優先度保証、deadline 保証。

根拠：[4] `kernel.h`、`kernel_process.c`、`kernel_trap.c`、[11] timer 実装。

PRELIMINARY

6 システムコール ABI と基本サービス

呼出し規約

PSC-OS の U → S ECALL は、**a3 に番号**、a0〜a2 に基本引数、サービスによって a4〜a6 にも引数を渡す。結果または status は a0、`SYS_ELF_RUN` では a1 にも終了コードを返す。MBIOS の S → M 呼出しは a7 を extension に使い、ユーザー ABI とは区別する。

```
/* PUTCHAR: 'A' */
register unsigned a0 __asm__("a0") = 'A';
register unsigned a3 __asm__("a3") = 1;
__asm__ volatile("ecall" : "+r"(a0) : "r"(a3) : "memory");
```

組込みユーザーコードは `src/api/user.h` と `user.c` のラッパーを用いる。文字出力のように dispatch が戻り値を設定しないサービスでは、a0 を成功コードとして解釈しない。入力待ちの数値 1000 はポーリング回数であり、1,000 ms の待ち時間を保証するものではない。

番号	名称	引数・動作
1	`SYS_PUTCHAR`	a0 の 1 文字を UART 出力。
2	`SYS_GETCHAR`	文字の受信まで待ち、a0 へ返す。PSC MMIO の受信関数は blocking。
3	`SYS_GETCHAR_TIMEOUT`	内部で `uart_getchar_timeout(1000)` を使用。未受信は − 1。
40	`SYS_DUMP`	a0=address、a1=length。0 なら 256 B、最大 4,096 B へ制限。
50	`SYS_SW_READ`	PIO 入力の下位 2 bit を返す。
100	`SYS_EXIT`	ELF 実行中は a0 の終了コードで復帰。通常シェルでは再起動。
101	`SYS_PRINT_INT`	a0 の整数を表示する。
102	`SYS_ELF_RUN`	a0=ELF バッファ、a1=ファイルサイズ。復帰時 a0=status、a1=終了コード。

外部 ELF へ公開する範囲

foreground ELF は番号 1、2、3、100、101 だけを呼べる。それ以外は a0 = − 1 を返す。ELF の実行中かどうかは `elf_is_running()` で確認し、通常 dispatch の前に制限する。シェルで未知の番号を呼ぶと PANIC へ進むため、定義済み番号でも dispatch の有無を確認する必要がある。

ヘッダに定義のみ存在する番号

`SYS_READFILE=51`、`SYS_WRITEFILE=52`、`SYS_LCD_INIT=90`、`SYS_LCD_FILL_RGB=91` には現行 dispatch の case がない。FAT32 読出しはユーザー側ライブラリと SD セクタ syscall で構成し、51 ／ 52 をファイルサービスとして呼び出さない。

番号と引数配置は Linux ／ POSIX ABI ではない。`user.c` に残る古い「ecall なし」というコメントより、実際の ECALL 命令と `kernel_syscall.c` の dispatch を優先する。

根拠：[7] `src/kernel/syscall.h`、`kernel_syscall.c`、`src/api/user.c`、`docs/psc_os_api.md`。

PRELIMINARY

7 デバイス系システムコール

番号	名称（SYS_接頭辞を一部省略）	引数・結果
10	SA_RUN	a0=A、a1=B、a2=C、a4=寸法、a5=option、a6=任意 profile。第 12 節参照。
20 / 21	I2S_MIC_READ/WRITE	a0=サンプル数。write は a1 も使用。下位 API の結果を返す。
30	SD_READ	a0=LBA。診断用セクタ読出し。
31	SD_WRITE_TEST	a0=LBA。512 B の試験パターンを書き込む。
32	SD_WRITE	a0=LBA、a1=512 B バッファ。status を返す。
33	SD_READ_BUF	a0=LBA、a1=512 B 出力先。成功時のみ内部バッファをコピー。
60	SPEECH_RECOGNITION	音声認識デモを実行し、結果を返す。
70 / 71	TIMER_START/START_AUTO	a0=reload。単発／自動リロード開始。
72	TIMER_STOP	タイマー停止。
73〜75	TIMER_GET_COUNT/GET_STATUS/IS_RUNNING	カウント、status、稼働状態。
76 / 77	TIMER_WAIT_US/WAIT_MS	a0 で µs ／ ms 単位の待機量を指定。
78 / 79	TIMER_MEASURE_BEGIN/END	計測開始／終了。終了結果は ms、利用不能は − 1。
87 / 88	TIMER_MEASURE_END_US/MEASURE_READ_US	µs 単位の終了／非破壊の途中読出し。
80	LED_WRITE	a0 を LED 出力値として設定。
81〜83	LED_ON/OFF/TOGGLE	a0=LED 番号。
84〜86	LED_ALL_ON/ALL_OFF/GET_STATE	全点灯、全消灯、状態取得。
92	LCD_RGB888_BEGIN	RGB888 描画モードを初期化。status を返す。
93	LCD_RGB888_RECT	a0=x、a1=y、a2=幅、a4=高さ、a5=RGB888 入力。

ポインタと同期実行

LCD 矩形は最大 16×16 画素、768 B であり、座標とユーザーページを検査後、カーネルスタックの一時バッファへコピーして転送する。転送終了後にユーザーポインタを保持しない。NPU も内部の物理バッファを介した同期実行で、成功時に出力をコピーする。

既存 SD read/write のポインタ取扱いは LCD ／ NPU と同一ではない。番号 33 は 512 B の内部バッファを経由するが、全ページ検査は実装されていない。このような診断・デバイスサービスを含むシェルは、信頼した組込みユーザーコードとして扱う。

エラー契約

戻り値の意味は各 API に従い、全サービス共通の errno 領域はない。NPU の引数異常／ timeout ／ busy、JPEG・FAT32・ELF のエラーはそれぞれ独自の番号体系である。呼出し側は status を確認した後に出力データを使用する。

根拠：[7] syscall ヘッダと dispatch、[11] ドライバ、[12] sa_transfer.h。

PRELIMINARY

8 UART シェルと操作体系

シェルは PSC_OS> で入力を受け、空白と tab でトークンを分割する。入力バッファは 32 B、文字列本体は最大 31 B、引数配列は最大 8 要素である。CR ／ LF、Backspace ／ DEL を扱う。引用符、パイプ、リダイレクトを備えた汎用シェルではない。

コマンド	動作
hello、primes[max]	コンソール確認、指定上限までの素数デモ。
dump[addr][len]、hexdump	メモリ診断。引数解釈は各分岐に従う。
sa_start、self_test	SynapEngine ／組込み診断。
sd_read、sd_write	セクタ読出し／試験書込み。通常ファイル操作とは別。
fat32_info、fat32_ls	ボリューム情報、root ディレクトリ一覧。
fat32_catFILE.TXT	ファイル内容をコンソールへ表示。
fat32_readFILE.TXT	最大 512 B を読出してサイズ・内容を表示。
fat32_touchFILE.TXT	既存実装の範囲で空ファイルを作成。
runHELLO.ELF	外部 ELF を同期実行し、結果を表示。
jpegTEST.JPG	Baseline JPEG の原寸 LCD 表示。
tflite_infoMODEL.TFL	モデルの構造・量子化・対応条件を確認。
tflite_runMODEL.TFLauto	CPU ／ NPU ／ PULP ／ AUTO を選択してデモ入力で推論。
tflite_benchMODEL.TFLquick	backend 比較。quick は比較対象を絞る。
tflite_diagon / off	推論外で出す診断情報の制御。
mic_read、mic_write、speech	マイク入出力診断、音声認識デモ。
micropython / microPython	組込み MicroPython REPL。Ctrl+D でシェルへ戻る。
help、exit / quit / q	ヘルプ、OS の再起動。

推奨する操作確認の順序

起動後は hello で UART、fat32_info と fat32_ls で SD、runHELLO.ELF で実行と復帰を確認する。画像は jpegTEST.JPG、推論は tflite_infoMODEL.TFL から進める。書込み診断は指定 LBA の内容を変更するため、読み出し確認の手順と分ける。

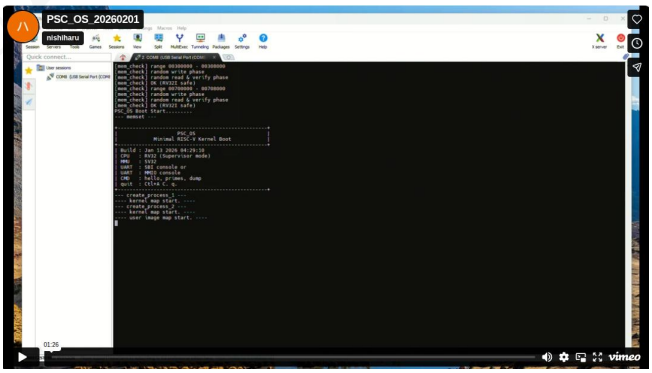


図 4: 過去の OS デモ画像にある UART 起動ログ。現行のコマンド一覧・ビルド番号は本文を基準とする。

根拠：[5] src/shell/shell.c の入力処理とコマンド dispatch。

PRELIMINARY

9 SD カードと FAT32 ファイルシステム

構成と対応範囲

FAT32 は U-mode 側ライブラリで、SD コントローラへの 512 B セクタ I/O を syscall 経由で呼び出す。JPEG・ELF・TFLite の stream 経路は、MBR 先頭パーティションの FAT32、512 B セクタ、root 直下、大文字 8.3 名を前提とする。`MODEL.TFL` は TFLite のバイト列を 8.3 名に収めたもので、形式変換を意味しない。

API	契約
`fat32_mount()`	ボリューム情報を読み、partition ／ FAT ／ data 基点を設定。
`fat32_find()`、`fat32_read()`	8.3 名で検索し、指定した上限まで読み出す既存 API。
`fat32_ls()`	root の全 sector と cluster chain を走査して一覧表示。
`fat32_open()`	名前・BPB を検査して root 内のファイルを開く。
`fat32_stream_read()`	現在位置から読出し。status=0 かつ短い読出しは正常 EOF。
`fat32_skip()`	前方向へ読み捨て。途中の SD エラーも検出。
`fat32_close()`	状態を無効化。動的メモリの解放や close syscall は不要。

LFN の表示と名前解決

一覧は最大 255 UTF-16 コード単位の LFN を復元し、UTF-8 で出力する。連番、checksum、type、cluster 欄、終端と padding を検証する。不正・欠落した LFN は 8.3 名へ戻す。サロゲートペアを扱い、不正なサロゲートは U+FFFD へ置き換える。

表示名は 28 桁幅と空白 2 つに整え、長い名前は UTF-8 文字を分断せず ... で省略する。sector ／非連続 cluster の境界をまたぐ LFN も対象である。ただし **open ／ find の名前指定は 8.3 のまま**であり、一覧に日本語名が出ても、その名前で開くことはできない。

逐次読み込みと破損検出

`fat32_file_t` は RV32 で 580 B であり、512 B のセクタバッファ、位置、サイズ、cluster 状態、循環検出状態を含む。JPEG の圧縮ファイル全体を RAM へ置く必要はない。stream API は不正 cluster、途中 EOC、Brent 方式の循環検出と volume 由来の上限を扱う。負のエラーは close ／ open まで保持する。

status	意味
0	成功。実読出し数を併せて判定。
− 2 / − 3 / − 4	名前不正／未発見／ SD I/O エラー。
− 5 / − 6 / − 7	形式不正／ cluster chain 異常／状態不正。

書込み機能の制限

`fat32_touch()` は既存の root 先頭 sector に空ファイルを作る範囲にとどまる。一般的なファイル更新、削除、ディレクトリ管理、VFS は提供しない。listing 側の破損 chain 対策も stream API とは異なり、複数 cluster 循環検出まで同等ではない。SD RTL には LBA0x1000 以下への書込み保護があり、試験用 volume 配置でも考慮する。

根拠：[8] `src/fs/fat32.h`、`fat32.c`、`fat32_stream.c`、`tests/fat32/README.md`。

PRELIMINARY

10 外部 ELF32 アプリケーション

受付形式とリソース上限

項目	仕様
ファイル	little-endian ELF32、RISC-V、ET_EXEC、ELF flags=0。
上限	ファイル 32 KiB、program header 8 個、PT_LOAD の実ページ 16 枚（64 KiB）。
配置	VA `0x00400000`〜`0x004fafff` のうち必要なページのみをマップ。
guard ／ stack	`0x004fb000` の 1 ページは未マップ。`0x004fc000`〜`0x004fffff` が 16 KiB の U+R+W、X なしスタック。
entry	整列済みで、実行可能な file-backed 領域の内側。
禁止する構造	重複する PT_LOAD ページ、W のみの権限、PT_DYNAMIC ／ INTERP ／ TLS。

ロードから復帰まで

1. `cmd_run_elf()` が FAT32 stream で最大 32 KiB を読み込む。シェルの既存 128 KiB スタックを入力用に利用する。
2. `SYS_ELF_RUN` でカーネルへ渡し、入力全ページと ELF の全配置計画を検証する。section header には依存しない。
3. カーネル所有ページをゼロ初期化し、各 PT_LOAD の `p_filesz` だけをコピーする。`p_memsz-p_filesz` とページ余白はゼロを保つ。
4. 元シェルの trap frame と root を保存し、専用 root へ切り替える。コード同期後、entry と SP を設定して U-mode で実行する。
5. EXIT なら status=0 と終了コード、処理可能な fault なら `PSC_ELF_FAULT=-11` を返し、シェルの SATP と context を復元する。

専用領域は root ／ leaf 各 1 ページとデータ・stack 用 20 ページ、計 88 KiB のカーネル BSS である。物理配置はリンク結果で決まり、ABI の固定 PA ではない。現在のプロセス ID とカーネルスタックを保ったまま一時実行するため、新しい scheduler process の生成ではない。

C++ サンプルの開発

```
# PSC_RV32I repository root
make -C PSC-ONE/software/os/elf_apps
make -C PSC-ONE/software/os/elf_apps inspect
# Copy elf_bin/HELLO.ELF, ARITH.ELF, LOOPTEST.ELF to SD root.
PSC_OS> run HELLO.ELF
PSC_OS> run ARITH.ELF
PSC_OS> run LOOPTEST.ELF
```

`extern"C"intuser_main()` が入口で、既存 start コードが戻り値を EXIT へ渡す。libc、STL、例外、RTTI、global constructor、GP ／ TLS 初期化は提供しない。relocation、PIE、共有ライブラリ、引数・環境変数、kill ／ timeout は未対応であり、無限ループする ELF は foreground を占有する。

根拠：[9] `elf_loader.h/.c`、`kernel_elf.c`、`tests/elf/README.md`、[10] `elf_apps/README.md`。

11 周辺デバイスと時間・音声サービス

PSC ドライバが使用する主な MMIO

デバイス	先頭アドレス	レジスタ／用途
UART	0x10000000	+0 TX、+4 RX、+8 status。TX busy bit 0、RX available bit 1。
PIO	0x10001000	switch の下位 2 bit をシェルがポーリング。
Timer	0x10002000	+0 control、+4 count、+8 status。
LCD	0x10003000	+0 pixel data、+4 status。ILI9488 用の下位転送。
LED	0x10004000	LED 出力制御。
SynapEngine	0x10005000	OS の MMIO マッピング対象。実行設定には専用 CSR も使用。
SD card	0x10006000	+0 data、+4 sector、+8 interface control。
I2S microphone	0x10007000	+0 受信、+4 status。
I2C slave	0x10009000	+0 input、+4 status、+8 output、+C control、+10 sequence。

タイマーと計測

タイマーは 1 count=1 µs を前提とする 16-bit reload を使い、status の running ／ autoreload ／ IRQ enable ／ IRQ pending は bits 7〜10 である。長い計測は 50 ms ごとの Machine IRQ で延長し、開始から終了までの時間を返す。タイマー使用中は計測を横取りせず、利用不能を返す。CLK_FREQ と実クロックの一致が必要で、50 ms 以上の IRQ 抑止や約 71 分を超える計測は対象外である。

I2C slave API

`i2c_api.h` はカーネル／ bare-metal 向けで、ECALL や MicroPython には公開しない。外部 master は 7-bit アドレス 0x42 へ INT8 入力 4 個を書き、output_ready を待って INT8 結果を読む。`i2c_wait_input(max_polls)` の上限は時間ではなくポーリング回数である。単体 RTL の既定アドレス 0x10002000 と SoC の I2C 配置を混同しない。

音声デモ

`speech` は I2S の録音、特徴抽出、分類を呼び出す組込みデモである。資料の画面には UP ／ DOWN ／ UNKNOWN の分類結果が示される。TFLite のモデル実行 API とは別の処理経路であり、この画像から認識精度や一般語彙への対応を保証しない。

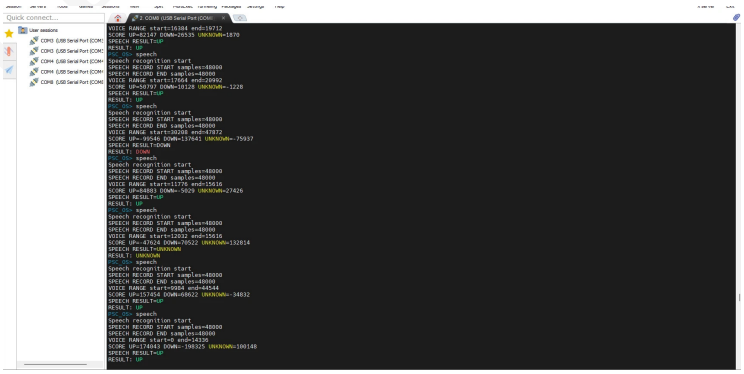


図 5: 既存の音声認識デモ画面。録音と分類結果の表示例。

根拠：[11] `src/drivers/`、`src/api/i2c_api.h/.c`、`speech_recognition_api.c`。

PRELIMINARY

12 DMA 転送と SynapEngine API

プロセスイメージの DMA コピー

KERNEL_PROCESS_USE_DMA=1 が既定で、memcpy_dma() は 4-byte 境界の 32-bit ワードを DMA で同期転送する。非整列・4 B 未満・末尾の 1〜3 B は CPU でコピーし、0 B では何もしない。対象は PA=VA の RAM で、重複領域や同時アクセス、DMA の競合使用は対象外である。

DMA は write-back D-cache を迂回するため、開始前に書戻しを完了し、終了後には無関係な dirty データを保存してから invalidate する。status の bit 1 は busy、bit 0 は done であり、busy を完了と解釈しない。USE_SBI_CONSOLE 構成では CPU コピーを使う。

CSR	DMA	CSR	SynapEngine
0x7e0	CTRL	0x7c0	CTRL
0x7e4	WORDS	0x7c4	SIZE
0x7e8	SRC	0x7c8	STATUS
0x7ec	DST	0x7d0	ADDR_A
0x7f0	STATUS ／ cache 制御	0x7d4/0x7d8	ADDR_B ／ ADDR_C

正方行列と長方行列

現行 sa_run_rect_checked() は $A[M,K]\,B[K,N] = C[M,N]$ を扱う。A ／ B は 8-bit、C は 32-bit、signed option を備える。SA_MAT_MAX=64、M は 1〜64、K ／ N は 4 の倍数で 4〜64 である。sa_run_checked() は正方形の互換 API である。

SYS_SA_RUN の a5 bit 30（RECT）を立てると、a4 は K[7:0]、N[15:8]、M[23:16] を格納する。RECT なしでは a4 を正方形の一辺と解釈する。a5 bit 0 は signed、bit 31 は profile 有効、a6 は profile 出力先である。旧呼出しの a5=0 ／ 1 では a6 を参照しない。

```
/* a4 encoding for rectangular transfer */
PSC_SA_DIMS(m, k, n) = (m << 16) | (n << 8) | k
/* a5 = PSC_SA_RECT_FLAG | signed_option
 * optionally PSC_SA_PROFILE_FLAG, with a6 = profile pointer */
```

転送・整合性・失敗時の動作

カーネルは寸法、C の 4-byte 整列、ユーザーバッファ全ページを検査し、A ／ B を内部物理バッファへコピーする。成功時だけ C をユーザーへ戻す。戻り値は 0=成功、− 1=引数、− 2=timeout、− 3=busy。timeout は controller を reset する。

通常 DMA とは異なり、現行 SynapEngine 経路は CPU と共通の data cache 経路を使う。ユーザー VA を NPU へ直渡ししないこと、完了待ちと既存 fence を維持することが前提となる。別 AXI DMA 経路へ接続する構成では、同じ整合性の前提を流用できない。

根拠：[12] dma.c、tests/dma/README.md、synap_api.h/.c、sa_transfer.h、[7] NPU dispatch。

PRELIMINARY

13 JPEG デコードと LCD 表示

対応する入力

`jpegTEST.JPG` は SD root の Baseline JPEG を、左上原点で 480×320 横向き LCD へ原寸表示する。画像全体やフルフレームバッファを RAM へ置かず、LCD 内部 GRAM を表示保持に利用する。画像外の領域は塗りつぶさない。

項目	条件
形式	Baseline DCT、8-bit、Huffman、単一 scan。
色・sampling	YCbCr 4:4:4、横 4:2:2、4:2:0、グレースケール。
寸法	幅 1〜480、高さ 1〜320。縮小指定 0（原寸）。
ファイル指定	FAT32 root、大文字 8.3 名。英大文字・数字・_・- の部分集合。
検査上限	ヘッダ走査 128 KiB、表 segment 512 B、decoder 作業領域 4 KiB。
未対応	Progressive、CMYK ／ YCCK ／ RGB JPEG、12-bit、Lossless、複数 scan、縦 4:2:2、EXIF 自動回転、リサイズ。

データ経路

```
FAT32 stream (512 B sector buffer)
  -> JPEG header validation
  -> TJpgDec / RGB888 MCU rectangle
  -> SYS_LCD_RGB888_RECT (max 16 x 16)
  -> kernel snapshot (max 768 B)
  -> ILI9488 window + 3 bytes/pixel
```

PSC 側で segment 長、表 ID、Huffman 記号などを事前検査してから TJpgDec へ渡す。ヘッダを再読するが圧縮画素 stream は一度だけ読み、adapter は EOI 終端も確認する。途中のエラーではシェルへ戻り、既に転送した部分画像は残る。

`jpeg_display.h` の現行設定は幅 480、高さ 320、`MADCTL=0xe8` である。初期資料中の `0x88` は現行値ではない。text 側の縦 320×480 定義は独立し、JPEG でソフトウェア回転を行わない。RGB888 矩形を転送し、RGB565 変換は行わない。

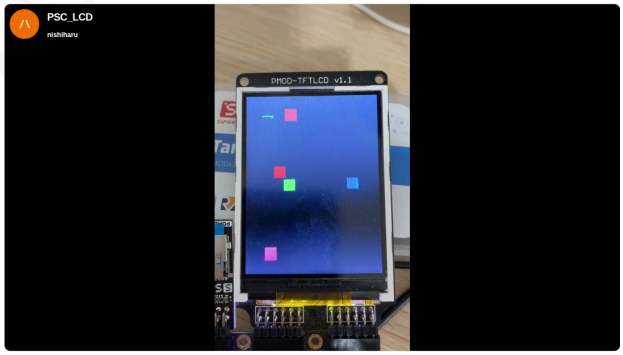


図 6: 既存の LCD 描画デモ（縦表示）。JPEG 横表示の検証写真ではない。

メモリと表示時間

JPEG session は既存報告で 4,824 B（work 4,096 B、FAT32 状態 580 B、JDEC 132 B ほか）。LCD 用 768 B は呼出し中の stack 領域である。表示時間は open から LCD 完了までを含み、timer 使用中は表示だけを行って計測不能と出力する。実機の色順序・方向・端の欠けは、`COLORS.JPG` や 17×19 の境界画像で確認する。

根拠：[13] `software/os/README.md`、`src/api/jpeg/`、`jpeg_display.h`、`lcd_api.c`。

14 TFLite INT8 推論ランタイム

モデルの受付と実行条件

TFLite 対応は限定された INT8 fully connected（FC）グラフの同期 runtime であり、TFLM インタプリタ全体ではない。`tflite_info` は構造検査と表示だけを行う。Profile compatible の表示は、arena 容量や accumulator 上限を含む prepare の成功を保証しない。

リソース／機能	現行の条件
モデル／ arena	モデル 8〜8,192 B、tensor arena 4,096 B。単一 resident model。
グラフ	1 SubGraph、入力 1・出力 1、Tensor 最大 32、Operator ／ opcode 最大 8、Buffer 最大 64。
shape	静的正整数、rank 1〜4、Tensor 最大 8,192 要素。FC は rank 2。現行 runtime は batch M を扱う。
Operator	FULLY_CONNECTED builtin v4、DEFAULT weights、fused NONE ／ RELU。
データ	INT8 入力・weight・出力、任意の INT32 bias。weight は constant で対称量子化。
量子化	per-tensor affine、正の有限 normal float32 scale。weight zero point=0、weight 値 − 128 を除外。
整合条件	bias zero point=0、bias scale は input scale×weight scale と 2 float32 ULP 以内。
未対応	Conv2D、動的 shape、per-axis 実行、sparse ／ variable tensor、custom operator。

準備・実行・所有権

```
int rc = psc_tflite_load("MODEL.TFL");
if (rc == 0) {
    size_t count;
    int8_t *input = psc_tflite_get_input(&count);
    /* Fill exactly count quantized INT8 input bytes. */
    rc = psc_tflite_set_fc_backend(PSC_TFLITE_FC_CPU);
    if (rc == 0) rc = psc_tflite_invoke();
    if (rc == 0) {
        const int8_t *output = psc_tflite_get_output(&count);
        /* Consume or copy output before load/reset/prepare. */
        (void)output;
    }
}
```

`load()` は失敗時にも旧 model を無効化する。`prepare()` は呼出し側が所有する 16-byte 整列・不変のモデルを借用し、reset ／ load ／ prepare までその寿命を維持する必要がある。input ／ output ポインタはこれらの操作や file inspection で失効し、output は成功した invoke の後だけ有効である。

呼出しは同期かつ非再入で、invoke 中の再入・backend 切替は BUSY となる。推論中に heap 割当や file I/O は行わない。モデルをロードしたら input を設定し、status を確認して output を利用する。

量子化の処理

raw dot に対して input zero point×weight row sum を補正し、INT32 bias、公式 TFLM 由来の二段丸め requantization、output zero point、INT8 ／ RELU clamp を順に適用する。CPU・PULP・NPU の違いは主に dot 積の実装であり、共通の後処理で最終結果を合わせる。

根拠：[14] `tflite_api.h`、`tflite_inspect.cc`、`tflite_runtime.cc`。初期 README の batch=1 制約は現行 AUTO 実装で拡張済み。

15 推論 backend 選択と性能資料の解釈

値	backend	振舞い
0	CPU	scalar FC。load ／ prepare ／ reset 直後の既定値。
1	SYNAP / npu	SynapEngine。明示指定の既定 tile=4、C API では 4 の倍数で最大 64。
2	PULP	signed `cv.dotsp.b` を利用。未対応ビルドや浅い K では CPU へ fallback。
3	AUTO	FC shape ごとに静的な選択規則を適用。推論中の試行 benchmark は行わない。

現行 AUTO 規則

`tflite_runtime.cc::select()` は、$M = K = N = 16$ または 32 の FC で NPU を選び、その一辺を tile とする。それ以外は PULP 有効かつ $K \geq 4$ なら PULP、利用不能なら CPU へ進む。モデル検査で拒否された Operator や量子化を、AUTO が別解釈で実行することはない。NPU のデバイスエラーも CPU へ自動再実行せず伝播し、output を無効化する。

現行 NPU adapter と資料の世代差

現在の adapter は A に実入力行、B に転置 weight を格納し、`psc_tflite_sa_tile()` へ $(M_{\text{実行}}, T, T)$ を渡す長方形経路を使用する。K ／ N 端部をゼロ padding し、実際に存在する M 行だけ計算・転送する。旧 Phase 4 や AUTO.md の正方形 padding の説明は、測定時点の方式として読む。

最大 workspace は INT8 64×64 を 2 面、INT32 64×64 を 2 面の合計 40 KiB（adapter と runtime の合計）。カーネル転送バッファは別であり、モデル上限 8 KiB と arena 上限 4 KiB は増えない。例えば 64×64 の入力と出力だけで arena を超えるため、tile 上限 64 を任意の 64×64 モデルの実行保証としない。

過去の OS RTL 測定例

下表は AUTO.md が記録する **CPU v1、100 MHz、legacy NPU、64 上限の校正ビルド**における $M = K = N = 32$ の値である。MMU、syscall、user ／ kernel コピー、整数後処理を含む。auto NPU16 追加前かつ現行長方形 adapter 以前の測定であり、本書対象ソースの再測定値ではない。

backend	engine 起動数	total invoke（µs）
CPU	0	67,970
PULP	0	32,687
NPU tile 4	512	168,791
NPU tile 16	8	38,879
NPU tile 32	1	26,247
NPU tile 64	1	70,684
AUTO（当該 shape で NPU32）	1	26,247

同報告の NPU32 は PULP 比 19.7% 短縮だが、最大 tile が最速とは限らない。小さい FC や padding の多い shape ではコピーと起動費用が支配する。性能比較は同一 firmware、同一入力、warm 条件、全出力一致、診断／ trace ／ profile 無効で行い、host mock 時間や bare-metal 時間を OS 実行時間と混ぜない。

根拠：[15] `AUTO.md`、`AUTO_REPORT.md`、`tests/tflite/auto_results.json`、[14] 現行 runtime ／ adapter。

PRELIMINARY

16 MicroPython 組込み環境

REPL と psc モジュール

`micropython` でシェル内の MicroPython を起動し、Ctrl+D で戻る。`importpsc` によりデバイス API と FAT32 上の Python スクリプト実行を利用する。独立した Python プロセスを生成する構成ではなく、シェルと同じユーザーイメージを共有する。

Python API	機能
`psc.fat32_ls()`、`psc.run(name)`	root 一覧と SD 上のスクリプト実行。
`psc.led_write(v)`、`led_on(n)`、`led_off(n)`、`led_toggle(n)`	LED 制御。各関数は psc モジュールに属する。
`psc.timer_start(n)`、`timer_start_auto(n)`、`timer_stop()`	timer 開始／停止。
`psc.wait_us(n)`、`psc.wait_ms(n)`	待機。`timer_count/status/running` で状態取得。
`psc.sa_run(...)`	既存 SynapEngine binding。
`psc.tflite_load(name)`、`tflite_reset()`	単一 resident model をロード／リセット。
`psc.tflite_input_size()`	入力 byte 数。未準備なら OSError。
`psc.tflite_run(data)`	入力をコピーして同期推論し、Python 所有の bytes を返す。
`psc.tflite_set_fc_backend(n)`	公開値は 0=CPU、1=Synap に限定。
`psc.tflite_set_synap_tile_size(n)`	公開値は 4、8、12、16 に限定。
`psc.tflite_arena_used()`	C 側 arena 使用量、reset 後は 0。

推論呼出し例

```
import psc
psc.tflite_load("MODEL.TFL")
size = psc.tflite_input_size()
# Example byte pattern only; supply correctly quantized data in applications.
data = bytes([0] * size)
psc.tflite_set_fc_backend(0)
result = psc.tflite_run(data)
print(list(result))
psc.tflite_reset()
```

型・所有権・例外

`tflite_run()` は読み取り可能な byte buffer を受け、text 文字列は拒否する。入力長はモデルが要求する byte 数と一致させる。INT8 は 2 の補数表現なので、− 1 は byte 値 255 となる。返却 bytes はコピーであり、後続の推論・load・reset を越えて保存できる。

型不正は TypeError、入力長／ backend ／ tile の不正やファイル名中の NUL は ValueError、C API 失敗は元の負値を `args[0]` に含む OSError となる。内部 arena への memoryview は公開しない。

C API の PULP ／ AUTO や tile 64 を、現在の Python binding から同じ範囲で選択することはできない。シェルと Python はモデルを共有し、`tflite_info` も旧モデルを無効化する。REPL 終了だけでは TFLite 状態を reset しない。

standalone の `micropython.elf` には PSC-OS の TFLite runtime をリンクしない。ここで示した 7 個の TFLite binding は OS 組込み構成の機能である。

根拠：[16] `software/micropython/ports/psc/README.md`、`modpsc.c`。

17 ビルド構成と検証手順

OS のビルド

Clang ／ LLVM、RISC-V libgcc 等の builtins、picolibc、MicroPython と TJpgDec の依存ソース、TFLite の固定 vendor を使用する。既定は `-march=rv32im_zicsr_zifencei-mabi=ilp32`。CPU_VERSION はソフトウェア機能も選ぶが、FPGA 内の CPU を切り替える操作ではない。

MODE	flow	USER_BASE	主な用途
`sim`	direct	`0x00400000`	シミュレーション用。BSS clear を省略する定義。
`fpga_mem`	direct	`0x00400000`	FPGA メモリイメージ。BSS clear 有効。
`psc`	loader	`0x00400000`	SD bootloader 経由の実機構成。
`sbi`	direct	`0x01000000`	OpenSBI 向けの別構成。PSC 配置と区別。

```
# Working directory: PSC_RV32I repository root
make -C PSC-ONE/software/os MODE=psc kernel_mem build/user.mem
make -C PSC-ONE/software/os fpga_fat32_boot
make -C PSC-ONE/software/os user_elf
# Document build: run in PSC_Docs/PSC_OS
make psc_os_doc
```

OS 生成物は `software/os/build/` へ出力する。ビルドは SD へ自動コピーしない。`SIM_USER=minimal` は検証用最小 shell であり、full shell の MicroPython・JPEG・TFLite を利用する構成とは区別する。QEMU を使う run target も PSC RTL の機能検証と同一ではない。

機能別の再現可能な確認

以下は参照資料にあるテスト入口である。本書作成時に新たに OS ／ RTL 全回帰を実行したという意味ではない。各 runner の依存環境と出力先を確認して利用する。

```
# Working directory: PSC_RV32I repository root
python3 PSC-ONE/software/os/tests/elf/run.py --build /tmp/psc-elf-check
python3 PSC-ONE/software/os/tests/fat32/run.py --build /tmp/psc-fat-check
python3 PSC-ONE/software/os/tests/jpeg/run.py --build /tmp/psc-jpg-check
python3 PSC-ONE/software/os/tests/jpeg/check_boundaries.py
python3 PSC-ONE/software/os/tests/tflite/check_vendor.py
python3 PSC-ONE/software/os/tests/tflite/run.py --build /tmp/psc-tfl-check
# RTL OS boot: working directory PSC-ONE/hardware/sim
make -f Makefile.pscos simulate_PSCOS CPU_VERSION=v1
```

合否の判定対象

ホスト試験では ELF 不正形式、FAT32 境界・LFN、JPEG の画素比較・異常入力、TFLite の量子化中間値・backend 一致を検査する。RTL では起動ログだけでなく `PSC_OS>` 到達、実行結果、fault 後の復帰、JUnit の FAIL 数を確認する。既存資料にも「TESTS=1 PASS=1 でも 0 prompts」の短時間実行があり、それをシェル起動完了とはしない。

実機では SD、UART、LCD 色順序・方向、I2S 入力、使用する CPU ／ NPU 構成の対応を別途確認する。標準 RTL Makefile は clean 呼出しを含み得るため、既存ログや成果物を残す必要がある場合は、専用 runner の分離した `--build` を利用する。

根拠：[2] OS Makefile、[17] 各テスト runner、`SRC_LAYOUT.md`、`hardware/sim/Makefile.pscos`。

18 実装範囲・参照資料・改訂履歴

現行実装の到達点

起動、U/S-mode 実行、Sv32 変換、UART、SD ／ FAT32、外部 ELF、JPEG、MicroPython、INT8 FC、DMA、SynapEngine 制御の実装が存在する。周期的プリエンプションは実装済みだが通常起動では無効である。完全な U/S メモリ隔離、汎用 VFS、network stack、POSIX process、動的リンク、任意 TFLite モデルの実行は提供範囲に含めない。

旧 raw-sector boot の `PSC_SD_ADDR` 未定義によるビルド失敗は既存資料の記録として残る。本書では FAT32 boot を主経路とし、raw-sector ／ SBI 実起動や現行長方形 NPU の性能を再検証したとは記述しない。リンク後の容量と性能は構成依存であり、過去値を固定仕様にしない。

参照資料

以下のパスは `PSC-ONE/` を基点とする。`src/` の省略形は `software/os/src/`、OS 内の `tests/` も同様とする。資料の更新日が異なる場合は、本文で示したコード上の現行定義を優先する。

番号	資料・コード
[1]	`docs/psc_os.md`、`docs/psc_os_api.md`、`software/os/README.md`
[2]	`software/os/Makefile`（MODE、ISA、ソース選択、依存関係）
[3]	`src/boot/psc_mbios.S`、`bootloader_fat32.c`、`boot_fat32.c`、boot 用リンカ
[4]	`src/kernel/kernel.c`、`kernel.h`、`kernel_process.c`、`kernel_trap.c`、`kernel_fpga.ld`、`user_memory.h`
[5]	`src/shell/shell.c`、`sim_shell.c`、`user.ld`
[6]	`docs/cpu_mmu.md`、`hardware/rtl/soc/cpu_v1/src/MMU.sv`
[7]	`src/kernel/syscall.h`、`kernel_syscall.c`、`src/api/user.h`、`user.c`
[8]	`src/fs/fat32.h`、`fat32.c`、`fat32_stream.c`、`tests/fat32/README.md`
[9]	`src/kernel/elf_loader.h/.c`、`kernel_elf.c`、`src/shell/elf_command.c`、`tests/elf/README.md`
[10]	`software/os/elf_apps/README.md`、同 `Makefile`、`syscall.hpp`
[11]	`src/drivers/` の UART 関連・SD・LCD・LED・timer・mic、`src/api/i2c_api.h/.c`、音声認識 API
[12]	`src/drivers/dma.c`、`tests/dma/README.md`、`src/api/synap_api.h/.c`、`sa_limits.h`、`sa_transfer.h`
[13]	`software/os/README.md`、`src/api/jpeg/`、`src/api/jpeg_display.h`
[14]	`src/api/tflite/` の公開ヘッダ、inspect ／ runtime ／ synap ／ platform 実装
[15]	同 `AUTO.md`、`AUTO_REPORT.md`、`PHASE4.md`、`tests/tflite/auto_results.json`
[16]	`software/micropython/ports/psc/README.md`、同 `modpsc.c`
[17]	`software/os/tests/`、`software/os/SRC_LAYOUT.md`、`hardware/sim/Makefile.pscos`
[18]	`docs/images/` の掲載 JPG 6 点、`LICENSE`、`software/os/third_party/README.md`

ライセンスと改訂

PSC-ONE 本体の LICENSE は MIT（Copyright 2026 QPSC-Design）。TJpgDec、MicroPython、TFLite 関連 vendor には個別のライセンスと通知があるため、配布時は各ソースの通知を保持する。

Rev.	日付	内容
0.8	2026-06-02	初期 Preliminary 仕様。
0.9	2026-10-10	現行コード・Markdown・JPG を統合。メモリ／ ABI を修正し、実装済み機能と制限、過去測定の条件を追記。

PRELIMINARY